



24 CI/CD + tests + versioning

US3.1 · C6

■ Objectifs & déroulé de la séance

À l'issue, vous serez capable de :

- › Écrire un workflow GitHub Actions (lint + test + build)
- › Configurer pre-commit (ruff, black, gitleaks)
- › Versionner données (DVC) et modèle (registry MLflow)
- › Gérer les secrets sans jamais les committer

Déroulé de la séance

- 1 **Théorie les concepts clés**
- 2 **Travaux dirigés** TP guidé, correction au fil de l'eau
- 3 **Autonomie tutorée** vous avancez, formateur dispo
- 4 **Bilan synthèse + preuves**

PREUVE FINALE Pipeline CI vert sur une PR · dvc status propre · 0 secret

1

Module 24 — les concepts clés

Théorie

■ Scénario atelier & enjeux

Vendredi 18 h, un push fait à la va-vite casse la prod du lundi — avec, au passage, un mot de passe oublié dans le code. Une usine automatique aurait tout bloqué.

Angle éthique / risque (C2) Un secret committé = une fuite de données. Le blocage des secrets (gitleaks) est un garde-fou réglementaire, pas un gadget.

- Éco-conception : une CI qui ne recalcule que le nécessaire (cache) consomme moins : pense au coût et au carbone des exécutions.

■ Contexte & outils — pourquoi ce module

Contexte : prouver la qualité à la main est fragile. Pourquoi : on automatise la preuve à chaque push et on versionne données + modèle.

- GitHub Actions (CI) : rejoue lint + tests + build à chaque push
- pre-commit : lance ruff/black/gitleaks AVANT le commit
- gitleaks : bloque un secret avant qu'il n'entre dans l'historique
- DVC : versionne les gros fichiers (pointeur dans Git + contenu sur un remote)
- MLflow : trace chaque entraînement + registre de versions du modèle

Objectif du module Une branche main toujours verte et un résultat reproductible : code + données + modèle.

■ La CI : un robot qui rejoue tout, à chaque fois

La CI relance automatiquement lint + tests + build à chaque modification, pour que la branche main reste toujours verte.



i

Le jargon décodé

CI = intégration continue. CD = livraison/déploiement continu. Pipeline = suite d'étapes automatisées.

≈

L'analogie

La CI, c'est le contrôle qualité en bout de chaîne : aucun produit ne sort sans avoir passé les tests.

EN UNE PHRASE main est un contrat : elle compile et passe les tests, toujours.

■ pre-commit & secrets : la première ligne de défense

pre-commit bloque les erreurs (et les secrets) AVANT qu'ils n'entrent dans l'historique Git.

- Hooks locaux : ruff, black, et gitleaks (détection de secrets)
- Un secret committé est compromis à vie : même supprimé, il reste dans l'historique
- Réflexe correct : RÉVOQUER la clé, pas seulement l'effacer

! Attention — piège

Effacer un secret d'un fichier ne le retire pas de l'historique Git. Il faut le révoquer.

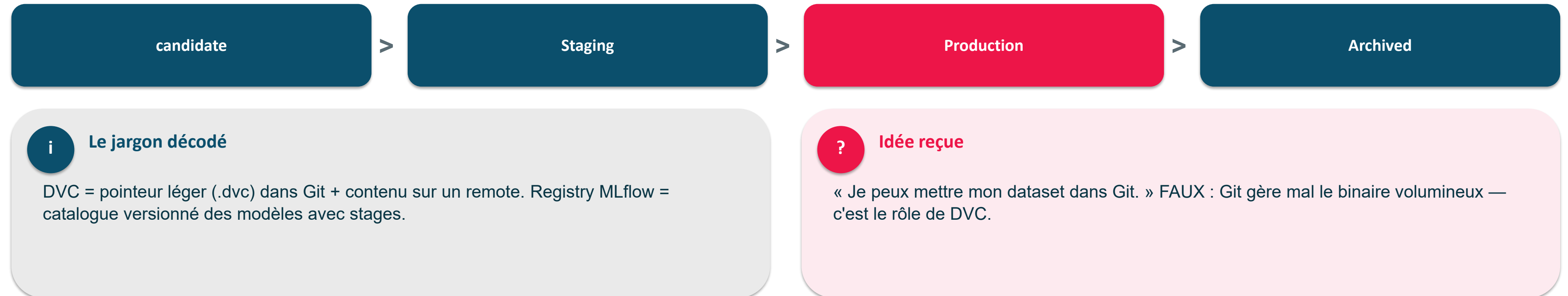
? Le saviez-vous ?

Des bots scannent GitHub en continu : une clé AWS poussée par erreur peut être exploitée en quelques minutes.

EN UNE PHRASE pre-commit local + gitleaks en CI = double filet contre les secrets.

■ Versionner données & modèle : DVC + MLflow

Git versionne le code ; DVC versionne les gros fichiers ; MLflow gère le cycle de vie du modèle.



EN UNE PHRASE Reproduire = même version de code + données + modèle, les trois ensemble.

■ Lire un log CI : par étape, pas en panique

Démo PR rouge vers verte : on repère l'étape qui a échoué, on corrige, on relance.

- Onglet Actions sur GitHub (ou démo locale : casser un test)
- Question : quel symptôme ? quelle étape a échoué ?
- Correction, puis `uv run pytest -q` (vert)

À retenir La CI ne remplace pas les tests : elle empêche juste d'oublier de les lancer.

EN UNE PHRASE Un log CI se lit étape par étape — chaque étape répond oui ou non.

■ Le job build : ne fabriquer que du code testé

Le squelette a un job quality ; on ajoute build, qui ne démarre que si quality passe.

```
build:
  needs: quality          # ne build que si les tests passent
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - uses: astral-sh/setup-uv@v3
    - run: uv build
    - uses: actions/upload-artifact@v4
      with: { name: indusense-wheel, path: dist/*.whl }
```

EN UNE PHRASE needs: quality = pas d'artefact à partir d'un code qui ne passe pas les tests.

■ Stratégie de versioning : code + données + modèle + secrets

- Code : PR + CI verte obligatoire
- Données : DVC (Git ne garde que le pointeur .dvc)
- Modèle : model_metadata.json, Candidate vers Staging vers Production vers Archived
- Secrets : .env local + secrets GitHub — jamais dans le code

```
uv run dvc init
uv run dvc add data/sample/capteurs_temperature.csv
git status
uv run dvc status
```



Idée reçue

« CI verte en local = verte sur GitHub. » Pas toujours : un test non hermétique (fichier local) passe chez toi et échoue en CI.

EN UNE PHRASE Reproduire un résultat = retrouver le code + les données + le modèle, ensemble.

■ À retenir en 3 points

- La CI teste et construit à chaque push.
- gitleaks bloque les secrets AVANT le commit.
- DVC versionne les données, MLflow versionne les modèles.

2

Module 24 — TP guidé, correction au fil de l'eau

Travaux dirigés

■ TP — workflow & pre-commit

```
# .github/workflows/ci.yml (extrait)
- uses: actions/setup-python@v5
  with: { python-version: "3.13" }    # PAS 3.11
- run: ruff check . && black --check . && pytest -q
```

✓ **Point de contrôle** Sur une PR : job quality puis build (needs). Un faux secret est bloqué par gitleaks.

■ TP — DVC & stratégie de versioning

```
uv run dvc init
uv run dvc remote add -d -f localstore ../dvc-store
git rm --cached -- data/gold/gold_dataset.csv
uv run dvc add data/gold/gold_dataset.csv
git add .dvc data/gold/gold_dataset.csv.dvc data/gold/.gitignore
uv run dvc push; uv run dvc status
```

- Documenter versioning_strategy.md (données + modèle + secrets)

■ TP — debug express (3 cas à corriger)

Symptôme

- CI verte en local, rouge en CI
- Build malgré des tests rouges
- Secret retiré du dernier commit

Cause / correction

- Test non hermétique (fichier local)
- needs: quality oublié
- Insuffisant : RÉVOQUER la clé

EN UNE PHRASE La plupart des échecs CI = test non hermétique, ou dépendance d'étape oubliée.

3

Module 24 — vous avancez, le formateur reste disponible

Autonomie tutorée

■ Autonomie tutorée — durcir la CI

Avancez à votre rythme. Le formateur reste disponible.

- Ajouter un job build qui upload le wheel en artefact
- OU enregistrer un modèle dans le registry MLflow (stage Staging)

Posture en distanciel

On compare les pipelines (vert/rouge) en fin de séance, écran partagé. Appelez à l'aide dès qu'un job reste rouge plus de 5 min.

■ Definition of Done & transition

- DoD : pipeline vert, versioning_strategy.md, 0 secret en clair

Transition → Module 25 Le package testé et versionné est prêt à être exposé en API REST.